LAZY PROPAGATION IN DATA STRUCTURES

Lazy Propagation is a technique mainly used with a **Segment Tree** to make **range updates faster**.

It is useful when we repeatedly need to:

1. **Update a range** of elements.
2. **Query a range** for sum, minimum, maximum, etc.

Instead of immediately updating every element in the range, we **delay the update** and store it in a lazy array.

That's why it is called **Lazy Propagation** — we are being "lazy" about updating the children until we actually need them.

---

1. SIMPLE EXAMPLE WITHOUT LAZY PROPAGATION

Suppose we have:

Array = [1, 2, 3, 4, 5]

Suppose we want to add 10 to every element from index 1 to 4.

That means:

[1, 2, 3, 4, 5]

↑ ↑ ↑ ↑

+10 +10 +10 +10

New array:

[1, 12, 13, 14, 15]

We updated 4 elements individually.

If the array contains:

1,000,000 elements

and we update:

500,000 elements

we may have to perform **500,000 updates**.

That can be slow.

---

2. IDEA OF LAZY PROPAGATION

Instead of immediately updating all children, we store:

"This entire segment needs +10."

For example:

Segment

[1 ..... 8]

|

+10

|

Don't update

children yet

We store +10 in a **lazy value**.

Later, if we need information from that segment's children, we pass the pending +10 to them.

This passing of the pending update is called:

Lazy Propagation.

---

3. SEGMENT TREE EXAMPLE

Consider:

Array = [1, 2, 3, 4]

A segment tree can look conceptually like:

[1,4]

/ \

[1,2] [3,4]

/ \ / \

1 2 3 4

Suppose we want:

Add 10 to [1,4]

The entire tree represents [1,4].

So instead of going to every leaf:

1 → +10

2 → +10

3 → +10

4 → +10

we simply record:

[1,4] → +10

and update the sum of that segment.

Original sum:

1 + 2 + 3 + 4 = 10

There are 4 elements.

After adding 10 to each:

New sum = 10 + (10 × 4)

= 50

So the root can immediately become:

50

while its children don't necessarily need to be updated yet.

---

4. WHAT IS THE LAZY ARRAY?

In a lazy segment tree, we normally maintain:

tree[]

lazy[]

tree[]

Stores the actual information for each segment.

For example:

tree[node] = sum of that segment

lazy[]

Stores a **pending update**.

For example:

lazy[node] = 10

means:

"Add 10 to every element represented by this node, but I haven't pushed this update to its children yet."

5. WHY DO WE NEED LAZY PROPAGATION?

Imagine:

Array = [1,2,3,4,5,6,7,8]

Now we perform:

Update [1,8] +10

Update [1,8] +20

Update [1,8] +30

Without lazy propagation, we might repeatedly update many nodes.

With lazy propagation:

lazy[root] = 60

because:

10 + 20 + 30 = 60

We don't need to immediately push all three updates down.

When a query later needs a child, we **push the pending 60 down**.

---

6. What Does "Push" Mean?

Suppose:

```
      [1,8]
       |
     lazy = 10
```

and its children are:

```
    [1,4]    [5,8]
```

When we need to access the children, we **push** the pending update:

```
      [1,8]
     lazy = 10
      /  \
     +10  +10
      ↓    ↓
```

[1,4] [5,8]

Then:

lazy[parent] = 0

The pending update has been passed to the children.

---

7. Real-Life Example

Imagine a teacher has **100 students**.

You say:

"Increase the marks of every student in this class by 5."

Normal approach

Go to every student:

Student 1 → +5

Student 2 → +5

Student 3 → +5

...

Student 100 → +5

100 operations.

Lazy approach

Write on the class board:

Entire Class → +5

You don't immediately change everyone's notebook.

When you need the marks of a particular group, you apply the pending +5.

That's the basic idea of lazy propagation.

---

8. Range Update Example

Consider:

Array:

Index: 0 1 2 3 4

Value: 1 2 3 4 5

Operation:

Add 10 to [1,3]

Expected array:

[1, 12, 13, 14, 5]

A segment tree doesn't necessarily update:

index 1

index 2

index 3

individually.

Instead, it identifies segments that are completely inside [1,3] and stores the pending update.

---

9. Query After Lazy Update

Suppose after:

Add 10 to [1,3]

we ask:

What is the sum of [1,3]?

The answer is:

12 + 13 + 14 = 39

The segment tree can calculate this using its stored segment information without necessarily visiting every individual element.

---

10. Time Complexity

This is the main reason Lazy Propagation is important.

Without Lazy Propagation

A range update can take approximately:

O(N)

because many elements may need to be updated.

With Lazy Propagation

Range update:

O(log N)

Range query:

O(log N)

So:

	Range Update	Range Query
Normal	O(N)	O(N)
Lazy Segment Tree	O(log N)	O(log N)

This is a major improvement for large data.

---

11. Important Terms

Term	Meaning
Segment Tree	Tree used for range queries/updates
Range Update	Update multiple elements at once
Lazy Array	Stores pending updates
Lazy Value	Update that hasn't been pushed down
Push	Pass pending update to children
Propagation	Moving the pending update down the tree

---

12. Easy Way to Remember

Think:

Normal Segment Tree

↓

Update everything immediately

Whereas:

Lazy Segment Tree

↓

Store the update

↓

Don't update children yet

↓

Update them only when needed

One-line definition

Lazy Propagation is a Segment Tree optimization that delays range updates and stores them in a lazy array until the affected child nodes need to be accessed.

The key idea is:

Range Update

↓

Update current segment

↓

Store pending update in lazy[]

↓

Don't immediately update children

↓

Push later when required

```
Algorithm LazyPropagation

Input:
    Array A
    Number of elements N

Create:
    Segment Tree tree[]
    Lazy Array lazy[]

Operations:
    1. Build Tree
    2. Range Update
    3. Range Query
```

```
BUILD(node, start, end)

1. If start == end:
       tree[node] = A[start]
       return

2. mid = (start + end) / 2

3. BUILD(left child, start, mid)

4. BUILD(right child, mid + 1, end)

5. tree[node] =
       tree[left child] + tree[right child]
```

```
BUILD(node, start, end)

    IF start == end THEN
        tree[node] = A[start]
        RETURN

    mid = (start + end) / 2

    BUILD(2 * node, start, mid)

    BUILD(2 * node + 1, mid + 1, end)

    tree[node] =
        tree[2 * node] +
        tree[2 * node + 1]
```